Architecture Review — `refinery_process_model`

Focus: deepening opportunities (locality + leverage). Generated: 2026-07-01.

Candidate A — Make `run_scenario` the deep module; push IO behind adapters

Strong

Core: treat `core.run_scenario` as the test surface; concentrate file loading + path resolution elsewhere.

Files

- `refinery_process_model/core.py`
- `refinery_process_model/io_inputs.py`
- `refinery_process_model/paths.py`
- `refinery_process_model/distillation.py`
- `refinery_process_model/hydrotreatment_`

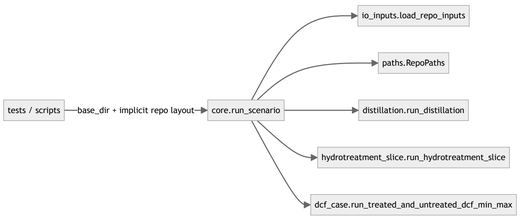
Problem

`run_scenario` is aiming to be deep, but still mixes algorithmic work with repo-specific file loading and path concerns. This reduces locality: tests have to know about TOML layout and base_dir.

Solution

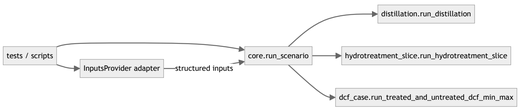
Create a seam: `InputsProvider` (interface) with adapters (`RepoTomlInputs`, maybe `InMemoryInputs`). Keep `run_scenario` mostly pure by requiring structured inputs and returning `ScenarioResult`.

Before (shallow seam)



Interface forces callers to care about filesystem layout and base_dir.

After (deep module + adapters)



Tests swap adapters; core stays stable and deep.

Benefits (locality + leverage)

- **Locality:** input loading rules live in one adapter; algorithm changes don't ripple into tests.
- **Leverage:** small interface (`ScenarioConfig` + structured inputs) buys many scenario permutations without file fiddling.
- **Tests:** fast in-memory tests can cover many scenarios; fewer Excel/TOML smoke tests needed.

Candidate B — Deepen the "slice" modules into a Pipeline module

Worth exploring

Unify distillation → DCF conversion → hydrotreatment into one orchestrating module with fewer tables leaking across seams.

Files

- `refinery_process_model/distillation.py`
- `refinery_process_model/distillation_sl`
- `refinery_process_model/hydrotreatment_`
- `refinery_process_model/cost_inputs.py`
- `refinery_process_model/dcf_case.py`

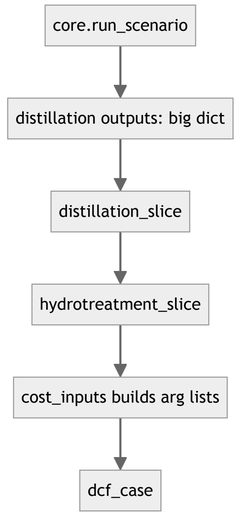
Problem

The "slice" modules return/accept wide dict-like tables. Callers must know lots of keys (interface is nearly as complex as implementation). Deletion test: deleting a slice likely spreads key-munging across callers.

Solution

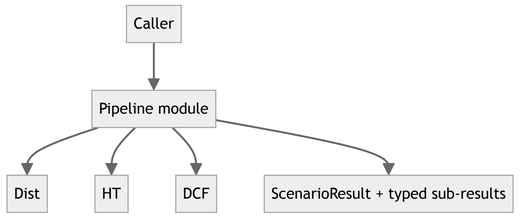
Introduce a Pipeline module that owns the ordering and table normalization. Keep "slice" modules as implementation details behind Pipeline's seam. The interface becomes "give config → get typed outputs".

Before



Many wide tables cross seams; callers need key knowledge.

After



Pipeline concentrates ordering + normalization, improving locality.

Benefits

- **Locality:** table-key knowledge lives in one module.
- **Leverage:** callers operate on typed results instead of dict keys.
- **Tests:** unit tests target Pipeline interface; fewer fragile key-level tests.

Candidate C — Replace ad-hoc test scripts with a Test Runner module

Speculative

Make tests themselves a deep module: one command, consistent fixtures, less sys.path hacking.

Files

- `refinery_process_model/tests_*.py`
- `refinery_process_model/test_premium_re`

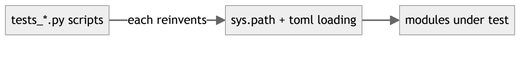
Problem

Tests are mostly standalone scripts (manual execution, duplicated setup, sys.path edits). This is a shallow interface for verification: knowledge of "how to run tests" is spread.

Solution

Create a single test-runner module (or adopt pytest fully) and a small fixture adapter layer (repo inputs vs in-memory). Keep scenario regression tests explicit.

Before



After

Benefits

- **Locality:** test setup in one place.
- **Leverage:** one command validates many modules consistently.
- **Tests:** clearer failures, parametrization, selective slow tests.

Top recommendation

Candidate A first: it strengthens the main seam (scenario execution) so everything else becomes easier to test and evolve. Once `run_scenario` is truly deep and mostly pure, Candidate B (Pipeline) and Candidate C (pytest) become much lower risk.